

Security Hardening Guide — Nucleus AI

Production security best practices and configurations.

1. CORS Configuration

In `backend/core/config.py`, ensure CORS is restrictive:

```
# Production CORS – only allow your domain
CORS_ORIGINS: list[str] = ["https://yourdomain.com"]
```

In `main.py`:

```
app.add_middleware(
    CORSMiddleware,
    allow_origins=settings.CORS_ORIGINS,
    allow_credentials=True,
    allow_methods=["GET", "POST", "PUT", "DELETE", "PATCH"],
    allow_headers=["Authorization", "Content-Type", "X-Request-ID"],
    max_age=86400, # Cache preflight for 24h
)
```

2. Rate Limiting

Backend (FastAPI)

```
pip install slowapi
```

```
from slowapi import Limiter, _rate_limit_exceeded_handler
from slowapi.util import get_remote_address
from slowapi.errors import RateLimitExceeded

limiter = Limiter(key_func=get_remote_address)
app.state.limiter = limiter
app.add_exception_handler(RateLimitExceeded, _rate_limit_exceeded_handler)

# Apply to specific endpoints
@app.post("/api/v1/workflow/execute")
@limiter.limit("10/minute")
async def execute_workflow(request: Request, ...):
    ...

@app.post("/api/v1/context/ingest")
@limiter.limit("30/minute")
async def ingest_content(request: Request, ...):
    ...
```

Nginx (already configured)

See `nginx/nginx.conf` for rate limiting zones.

3. Security Headers

Already configured in Nginx. Additional headers for the frontend:

```
// next.config.ts
const securityHeaders = [
  { key: 'X-DNS-Prefetch-Control', value: 'on' },
  { key: 'Strict-Transport-Security', value: 'max-age=63072000; includeSubDomains; preload' },
  { key: 'X-Frame-Options', value: 'SAMEORIGIN' },
  { key: 'X-Content-Type-Options', value: 'nosniff' },
  { key: 'Referrer-Policy', value: 'strict-origin-when-cross-origin' },
  { key: 'Permissions-Policy', value: 'camera=(), microphone=(), geolocation=()' },
  {
    key: 'Content-Security-Policy',
    value: "default-src 'self'; script-src 'self' 'unsafe-eval' 'unsafe-inline'; style-src 'self' 'unsafe-inline'; img-src 'self' data: https:; font-src 'self' data:; connect-src 'self' https://api.yourdomain.com;"
  },
];

module.exports = {
  async headers() {
    return [{ source: '/*.*', headers: securityHeaders }];
  },
};
```

4. Secrets Management

Best Practices

1. **Never commit secrets to git** — use `.env` files (in `.gitignore`)
2. **Use managed secret stores:**
 - AWS: Secrets Manager or SSM Parameter Store
 - GCP: Secret Manager
 - Azure: Key Vault
 - Self-hosted: HashiCorp Vault or Docker secrets
3. **Rotate secrets regularly** — especially API keys and database passwords
4. **Use different secrets per environment** — staging $\neq$ production

Generating Secrets

```
# Database password
openssl rand -hex 32

# Application secret key
openssl rand -hex 64

# NextAuth secret
openssl rand -base64 32
```

5. Database Security

```
-- Create read-only user for reporting
CREATE ROLE nucleus_readonly WITH LOGIN PASSWORD 'strong_password';
GRANT CONNECT ON DATABASE nucleus TO nucleus_readonly;
GRANT USAGE ON SCHEMA public TO nucleus_readonly;
GRANT SELECT ON ALL TABLES IN SCHEMA public TO nucleus_readonly;

-- Enforce SSL connections
ALTER SYSTEM SET ssl = 'on';
```

6. Network Security

- PostgreSQL and Qdrant should **only** be accessible from backend containers (bind to `127.0.0.1`)
- Use Docker networks to isolate services
- Enable UFW/iptables on VPS deployments
- Use VPC private subnets on cloud providers

7. Container Security

- Run containers as non-root users (already configured in Dockerfiles)
 - Use multi-stage builds to minimize attack surface
 - Scan images for vulnerabilities:

```
bash
docker scout cves nucleus-ai/backend:latest
# or
trivy image nucleus-ai/backend:latest
```
 - Pin base image versions (not `latest`)
 - Enable Docker Content Trust: `export DOCKER_CONTENT_TRUST=1`
-

8. API Authentication

The frontend uses NextAuth.js for user authentication. For API-to-API communication:

```
# backend/core/auth.py
from fastapi import Depends, HTTPException, Security
from fastapi.security import APIKeyHeader

api_key_header = APIKeyHeader(name="X-API-Key", auto_error=False)

async def verify_api_key(api_key: str = Security(api_key_header)):
    if not api_key or api_key != settings.API_KEY:
        raise HTTPException(status_code=403, detail="Invalid API key")
    return api_key

# Usage
@router.post("/api/v1/workflow/execute", dependencies=[Depends(verify_api_key)])
```

9. Security Checklist

- [] All secrets in environment variables or secret manager
- [] HTTPS enforced (HTTP redirects to HTTPS)
- [] CORS restricted to production domain
- [] Rate limiting enabled
- [] Security headers configured
- [] Database not publicly accessible
- [] Containers run as non-root
- [] Docker images scanned for vulnerabilities
- [] SSH key-only authentication
- [] Firewall configured (only 80, 443, 22)
- [] Automated security updates enabled
- [] Fail2ban configured
- [] Logs monitored for suspicious activity
- [] PII scrubbing enabled (Module D)
- [] Regular secret rotation scheduled